

LocalLLM

Architecture Overview — private, self-hosted multimodal AI platform

LocalLLM packages multimodal chat, document OCR, speech-to-text, real-time speech translation with spoken output, and a tool-using autonomous agent into a single self-hosted platform built around **Gemma 4 12B** served via **llama.cpp**. Everything runs on the user's own hardware — an NVIDIA GPU workstation (CUDA) or an Apple Silicon MacBook Pro (Metal) — with zero marginal cost, full data privacy, and offline operation. A one-line configuration switch can redirect the same client code to a commercial OpenAI-compatible API when desired.

Design principles

- **Single-gateway design.** Exactly one FastAPI service (*localllm-serve*) owns the only *llama-server* subprocess and exposes an OpenAI-compatible HTTP API. Every app is a thin client — no process ever loads the ~10 GB model twice, and concurrency is centralized.
- **Provider-agnostic client SDK.** Business logic talks to an *LLMClient* protocol, never to “local vs. cloud”. The same code runs against Gemma locally or GPT-class models in the cloud, selected purely by configuration.
- **Pipeline composition over monoliths.** Speech translation is a chain of small, testable, individually mockable stages (VAD segmentation → ASR → translation → sentence-queued TTS) that stream partial results.
- **Cross-platform GPU acceleration.** CUDA vs. Metal is auto-detected at runtime; all model layers are offloaded to the GPU by default, with VRAM-tuning profiles for constrained cards.
- **Privacy and safety by default.** Services bind to 127.0.0.1, agent file tools are sandboxed to the project directory, the web-fetch tool blocks private IP ranges (SSRF defense), and secrets stay out of version control.
- **Layered, typed configuration.** Pydantic-settings models layer YAML defaults, named profiles, and *LOCALLLM_*-prefixed environment overrides.

Capabilities at a glance

Capability	Entry point	What it does
Multimodal chat	localllm-chat / webui	Text + image + audio understanding
Web UI	localllm-webui (:8095)	React SPA: chat, agent mode, live translate, OCR, STT
Tool-using agent	localllm-agent	LangGraph ReAct agent, sandboxed tools, skills system
Document OCR	localllm-ocr	PyMuPDF text layer + local vision OCR for scans
Batch speech-to-text	localllm-stt	Overlap-aware 28 s chunking, local merge
Live voice translation	webui Live mode	Streaming VAD → ASR → MT → offline TTS
Inference gateway	localllm-serve (:8090)	OpenAI-compatible API over llama-server

System architecture

The diagram below shows the full stack. All inference flows through one shared gateway; client applications never talk to *llama-server* directly and never load model weights.

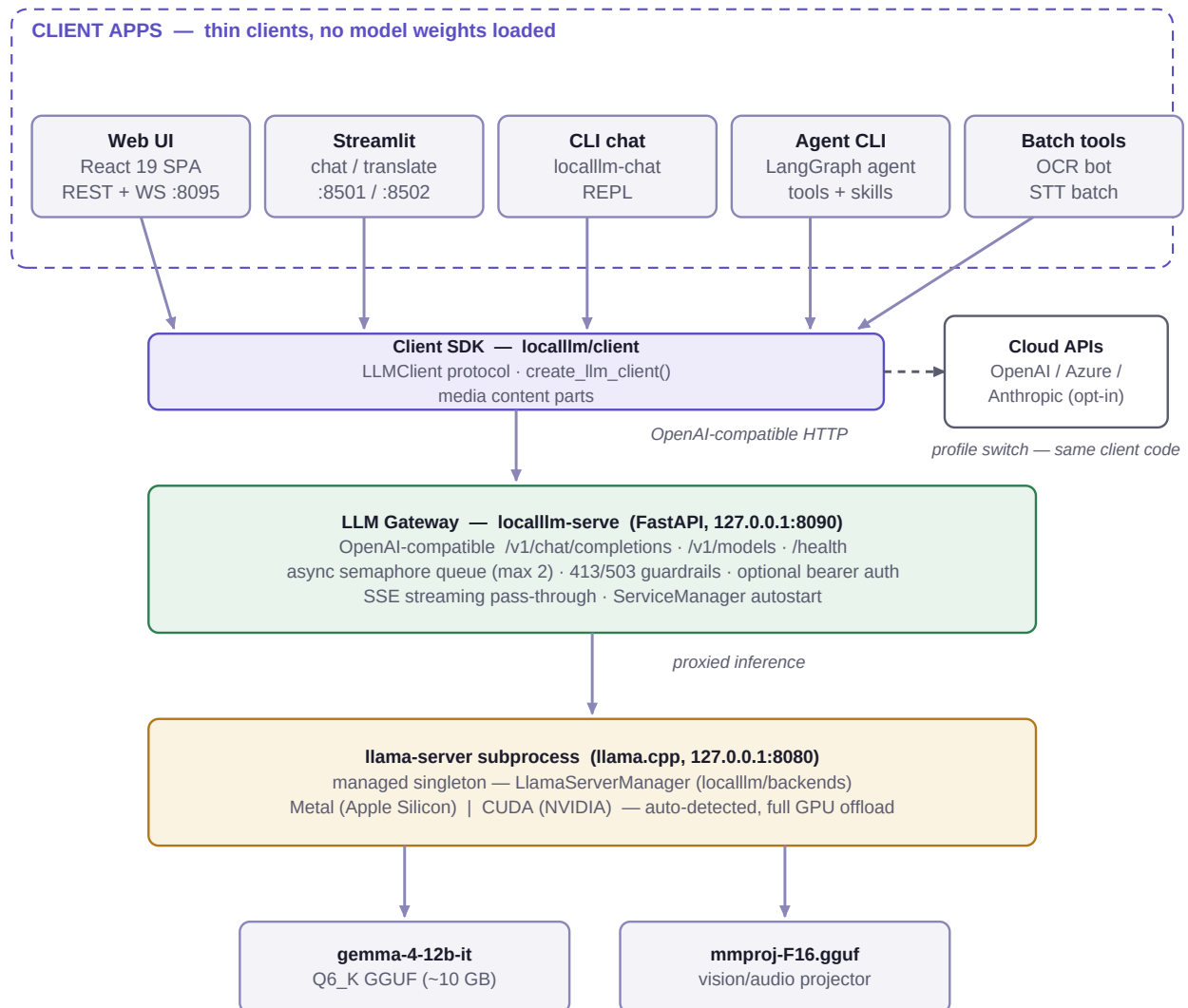


Figure 1 — LocalLLM service topology (default local ports)

Request lifecycle. (1) A client builds messages through the client SDK and POSTs to the gateway at `http://127.0.0.1:8090/v1/chat/completions`. (2) If the gateway is not running, `ServiceManager.ensure_running()` spawns it as a subprocess — apps self-heal after a crash. (3) On startup the gateway's lifespan hook starts the single *llama-server* subprocess and waits for it to become healthy. (4) The request acquires a slot in an async semaphore (default 2 concurrent; waiting requests time out with 503 after 30 s) and is proxied to llama-server, including Server-Sent-Events pass-through when `stream: true`. (5) Multimodal content is ordered images-before-text and audio-after-text to match Gemma's expected layout.

Component reference

Client SDK — `localllm/client`

`protocol.py` defines the `LLMClient` protocol; `openai_compatible.py` implements it for any OpenAI-compatible HTTP endpoint over `httpx`; `factory.py` (`create_llm_client()`) selects the implementation from configuration; `media.py` builds image/audio/text content parts. Every app — CLI, Streamlit, agent, OCR, STT, web UI — depends only on this layer, which is what makes the local/cloud switch a pure configuration change.

Gateway service — `localllm/service`

`app.py` is the FastAPI application behind `localllm-serve: /v1/chat/completions, /v1/models` and `/health`, with hardening guardrails — request-size cap (64 MB → 413), queue timeout (→ 503), upstream error mapping (502/504) and optional bearer-token auth for LAN exposure. `manager.py` (`ServiceManager`) lets any client autostart the gateway as a detached subprocess.

Inference backend — `localllm/backends` + `localllm/devices`

`llama_server.py` (`LlamaServerManager`) is a singleton that owns the `llama-server` process: builds the command line (model, mmpoj, context size, `n_gpu_layers=999`), waits for readiness, and tears it down on shutdown. `devices/resolver.py` detects the platform — Metal on Apple Silicon, CUDA on NVIDIA — so one config works on both target machines. The binary itself is an external dependency (Homebrew build ≥ 9590 on macOS, CUDA release build on Windows); Python only manages the subprocess and HTTP.

Applications — `apps/`

Console entry points declared in `pyproject.toml`: `cli_chat` (terminal REPL), `streamlit_chat` and the translate app (launched via subprocess wrappers to avoid nested Streamlit runs), `agent_cli`, `ocr_bot`, `stt_batch`, `serve`, `webui`, and `live_bench` — a benchmark harness for the live pipeline with a `--fake` mode that needs no GPU.

Web UI — `webui/` + `localllm/webui/server.py`

A React 19 + Vite + Tailwind v4 single-page app, committed as a prebuilt static bundle (`webui/dist`) so runtime needs no Node. The FastAPI server on `:8095` serves the bundle and a REST API: chat with agent mode and per-skill toggles, multi-file attachments, NDJSON-streaming translate/transcribe/TTS/OCR, health/meta — plus the `/ws/translate` WebSocket for live voice-to-voice translation (Figure 2).

Agent — `localllm/agents`

`graph.py` builds a LangGraph ReAct-style loop with a custom, fence-tolerant tool-call parser (Gemma's native function-calling wire format is a planned hardening step). `tools.py` provides sandboxed file tools (project-directory jail, secret-file denylist, extension allowlist), web search, and a page-fetch tool that blocks private/loopback IP ranges. A pluggable `skills/` directory adds discoverable task recipes.

Pipelines & media — `localllm/pipelines`, `localllm/media`, `localllm/tts`, `localllm/live`

OCR (PyMuPDF text layer, 150-DPI page rendering for scans, vision OCR with JSON-repair fallback), batch STT (28 s chunks, 2 s overlap — Gemma's hard 30 s audio limit), chunked translation, offline Piper TTS with per-language voices, and the streaming live-translation session described on the next page. Media helpers cover PDF, DOCX, images (Pillow) and audio (librosa/soundfile/ffmpeg).

Configuration — `localllm/config.py` + `config/`

Typed pydantic-settings models layered as: `config/default.yaml` → named profile YAML (e.g. an OpenAI profile) → `LOCALLLM_`-prefixed environment variables with `__` nesting. Key defaults: gateway `:8090`, llama-server `:8080`, web UI `:8095`, thinking mode disabled, model revision pinnable for supply-chain safety.

Real-time voice-to-voice translation

The live pipeline (*localllm/live*) turns continuous speech into translated speech a few seconds behind the speaker, entirely on-device. It is built from injectable stages so each can be unit-tested or faked without a GPU.

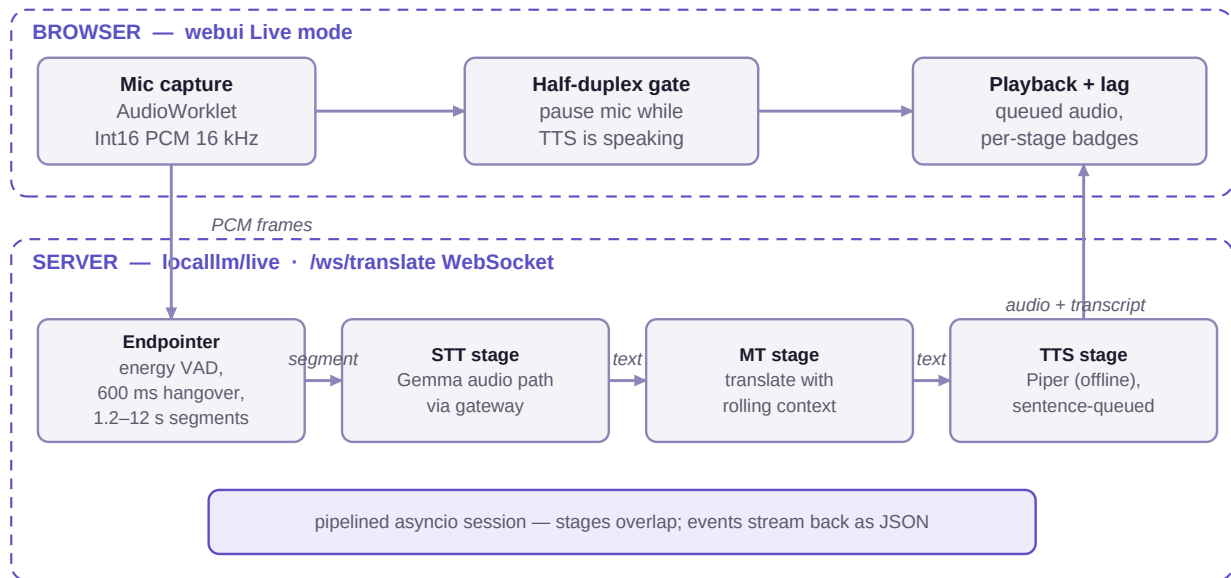


Figure 2 — streaming STT → MT → TTS session over the /ws/translate WebSocket

The browser captures microphone audio with an AudioWorklet and streams Int16 PCM frames over the WebSocket. A streaming **endpointer** (energy-based VAD with a 600 ms hangover) cuts the stream into 1.2–12 s utterance segments. Each segment then flows through a **pipelined asyncio session**: while one segment is being spoken by TTS, the next is already being transcribed — the stages overlap, which is what keeps steady-state lag within the ≤ 8 s target. Translation keeps a rolling context of previous sentences for consistency. Events stream back as JSON (*segment*, *transcript*, *translation*, *audio*) so the UI can show per-stage lag badges; a half-duplex gate pauses the microphone while synthesized speech plays to prevent feedback. **localllm-live-bench** replays a recording through the same code path and reports lag statistics (*--fake* swaps the model stages for stubs).

Cross-platform operation

The same repository runs on both target machines with no code changes: the device resolver picks **Metal** on Apple Silicon and **CUDA** on the NVIDIA workstation (RTX 5060 16 GB); platform-specific operational scripts ship in pairs (*scripts/*.ps1* for Windows, *scripts/*.sh* for macOS/Linux — GPU cleanup, stack startup, stale-port cleanup); and the web UI bundle is committed so a fresh machine needs only Python, the llama.cpp binary, and the downloaded GGUF files. If VRAM is constrained, profiles lower quantization (Q5_K) or *n_gpu_layers*.

Security posture

- **Network:** gateway and web UI bind to 127.0.0.1 by default; optional bearer-token auth (*service.api_key*) must be set before any LAN exposure.
- **Request guardrails:** 64 MB body cap (413), inference-slot queue timeout (503), upstream failure mapping (502/504) — a misbehaving client cannot wedge the model.
- **Agent sandbox:** file tools are jailed to the project directory with path-traversal checks, a secret-file denylist (e.g. *hf_token.txt*, *.env*), and an extension allowlist.
- **SSRF defense:** the web-fetch tool resolves and rejects private, loopback, and link-local address ranges before connecting.
- **Supply chain:** the Hugging Face download pins a model revision; secrets are read from *hf_token.txt* / *HF_TOKEN* and never committed by tooling.

Quality & testing

The unit suite — **143 tests** — runs with **no GPU and no model**: the client SDK, gateway guardrails and SSE proxying, agent parser and tool security, media handling, TTS, web UI API, and the live endpointer/pipeline/WebSocket are all exercised against mocks. Static quality is enforced with **ruff** (lint + import order), **mypy** (typed against Python 3.10), and **bandit** (security lint), wired into a GitHub Actions CI workflow. Integration tests that require a real llama-server are a planned, separately-gated tier.

Repository layout

Path	Purpose
apps/	Console & UI entry points (serve, chat, webui, agent, OCR, STT, bench)
localllm/client/	Provider-agnostic LLM client SDK
localllm/service/	FastAPI gateway + ServiceManager autostart
localllm/backends/	llama-server subprocess manager
localllm/agents/	LangGraph agent, sandboxed tools, skills
localllm/pipelines/	OCR, batch STT, translation pipelines
localllm/live/	Streaming endpointer + live translation session
localllm/media/, tts/	PDF/DOCX/image/audio helpers, Piper TTS
localllm/webui/	Web UI server (REST + WebSocket)
webui/	React SPA source + committed dist/ bundle
config/	default.yaml + provider profiles
scripts/	Model download, GPU/port cleanup (.ps1 + .sh pairs)
tests/	143-test unit suite (no GPU required)

Sources: *AGENT_HANDOFF.md*, *docs/RUNNING.md*, *README.md* and the *localllm* package, June 2026 (v0.2.0).